Faculty of Engineering
THE UNIVERSITY OF HONG KONG

ROS2 Tutorial – Dual-UR5 Cube Picking (Advance)

Conducted by:
Chenwei Wang, Yao Zhong, Zhongliang Jiang
MIRoC Lab, Faculty of Engineering, HKU
March. 2026

Goal and Contents

- **Goal**
  - Simulate and control a dual UR5e robotic system equipped with Robotiq grippers and an egocentric vision system to perform pick-and-place tasks.

- **Contents**
  - Prerequisites
    - Configuring environment
    - Background
  - Tasks
    - 1 Start the simulation environment
    - 2 Launch Demo 1
    - 3 Modify the trajectory of Demo 1 to pick up the cube.
    - 4 Impove Demo 1
    - 5 Use right arm to do the same thing
    - 6 Deploy Demo 2
    - 7 Deploy Demo 3

Configuration Environment

- ROS 2 relies on the notion of combining workspaces using the shell environment.
- “Workspace” is a ROS term for the location on your system where you’re developing with ROS 2.
- The core ROS 2 workspace is called the underlay.

Before we start, please make sure you have installed ROS2 correctly!

Clone the required files

```
git clone https://github.com/wwccww/ros2-advance
```

Open the ppt, then you can easily copy the command

Configuration Environment

Install gazebo, Moveit2

```
sudo apt update
sudo apt install gazebo
sudo apt install ros-humble-moveit*
sudo apt install ros-humble-gazebo-ros-pkgs
```

In the Linux terminal, it will ask you to enter password when you use sudo command.

The password is qtfuture

Install ros2 control

```
sudo apt install ros-humble-ros2-control
sudo apt install ros-humble-ros2-controllers
```

Configuration Environment

Install dependencies of image

```
sudo apt install ros-humble-image-pipeline
sudo apt install ros-humble-compressed-image-transport
sudo apt install ros-humble-compressed-depth-image-transport
sudo apt install ros-humble-vision-msgs
```

In the Linux terminal, it will ask you to enter password when you use sudo command.

The password is qtfuture

Clone this repo

```
git clone https://github.com/zitongbai/UR5e_Vision_Assemble
cd UR5e_Vision_Assemble
sed -i 's/git@github.com:/https:\/\/github.com\//g' .gitmodules
git submodule sync
git submodule update --init --recursive
```

Configuration Environment

Open another terminal(1) and Build the repo

```
cd ~/UR5e_Vision_Assemble
colcon build --symlink-install
```

What is Gazebo?

Gazebo is a 3D dynamic simulator with the ability to accurately and efficiently simulate populations of robots in complex indoor and outdoor environments. While similar to game engines, Gazebo offers physics simulation at a much higher degree of fidelity, a suite of sensors, and interfaces for both users and programs.

Typical uses of Gazebo include:

- testing robotics algorithms,
- designing robots,
- performing regression testing with realistic scenarios

A few key features of Gazebo include:

- multiple physics engines,
- a rich library of robot models and environments,
- a wide variety of sensors,
- convenient programmatic and graphical interfaces

GAZEBO

Video

What is MoveIt 2?

MoveIt 2: The Brain (Motion Planning)

•**What it is:** The premier motion planning framework for ROS 2.
•**Role in our project (Plan):** Solves complex Inverse Kinematics (IK) to figure out how to bend the joints to reach a target.

Continuously monitors the environment (like the table we added in the code) to generate **collision-free trajectories** for both arms simultaneously.

MoveIt

What is Yolo?

YOLOv5: The Eyes (Perception)

•**What it is:** A state-of-the-art, real-time object detection and segmentation model.
•**Role in our project (Sense):** Processes RGB-D data from the simulated RealSense camera.

Locates the target block and translates 2D pixel coordinates into 3D spatial frames (TF) for the robot to grasp.

Video

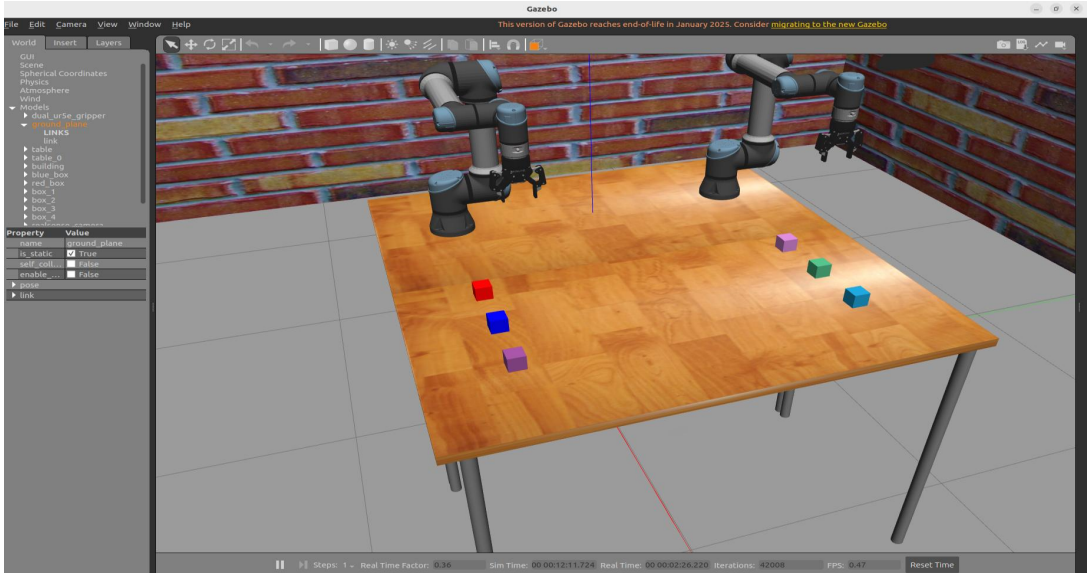
Task 1 Start the simulation environment

Start the simulation environment

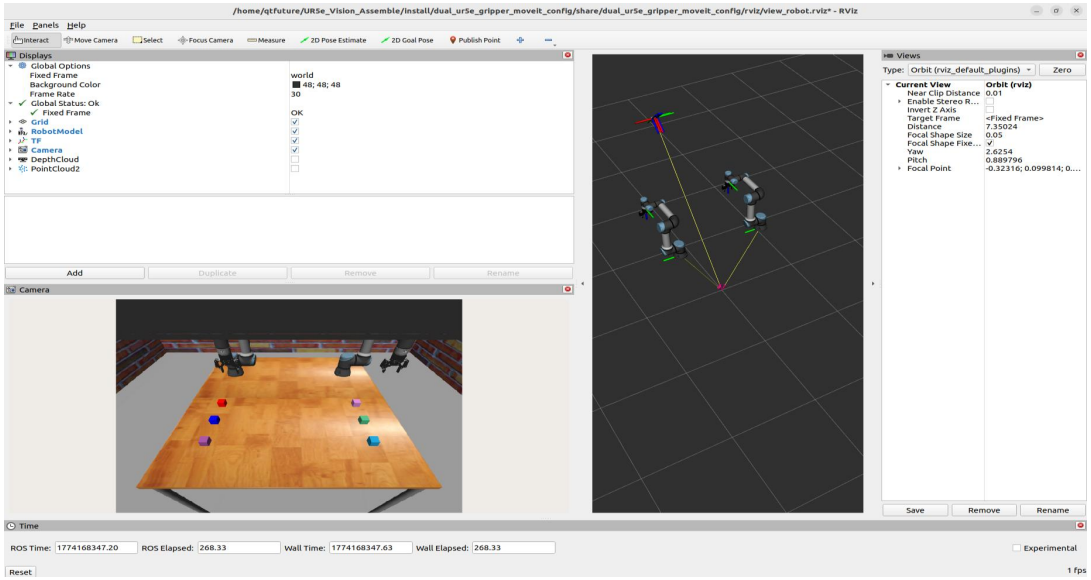
```
cd ~/UR5e_Vision_Assemble
source install/setup.bash
ros2 launch bringup simulation.launch.py
```

Task

Gazebo



Rviz



Task 2 Launch the Demo 1

- Open a new terminal:

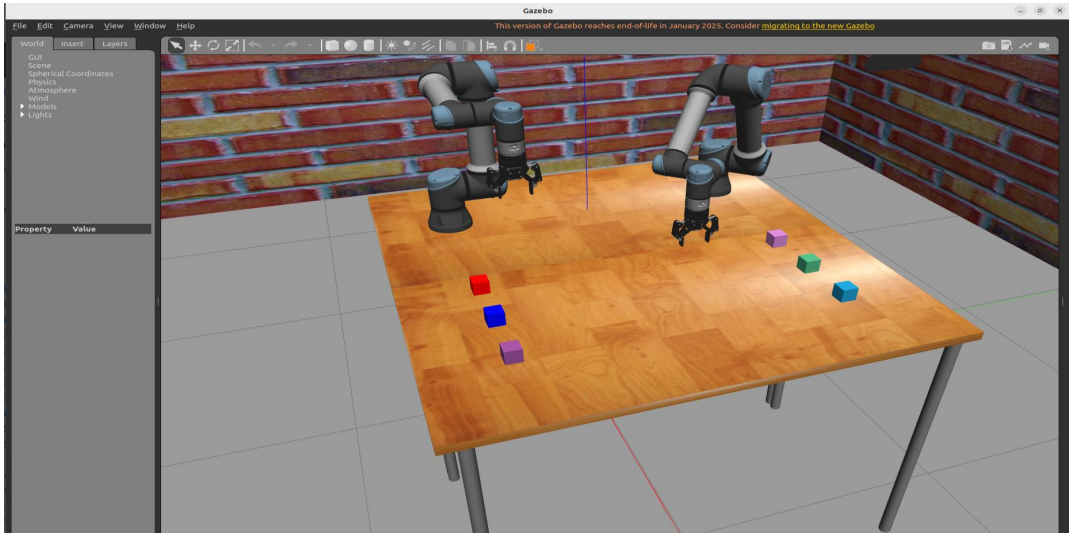
```
cd ~/UR5e_Vision_Assemble
source install/setup.bash
```

- Launch Demo 1

```
ros2 launch ur5e_gripper_control demo1.launch.py
```

Demo 1: The original demo1 serves as a motion planning baseline, executing a hardcoded sequence of waypoints and gripper actuations to verify the MoveIt2 control pipeline.

Task 2 Launch the Demo 1



Demo 1: The original demo1 serves as a motion planning baseline, executing a hardcoded sequence of waypoints and gripper actuations to verify the MoveIt2 control pipeline.

Task 3 Modify the trajectory of Demo 1 to pick up the cube

- Open the project folder in VS Code
- Open src/ur5e_gripper_control/launch/demo1.launch.py
- Modify the target_position value as shown in the figure below.

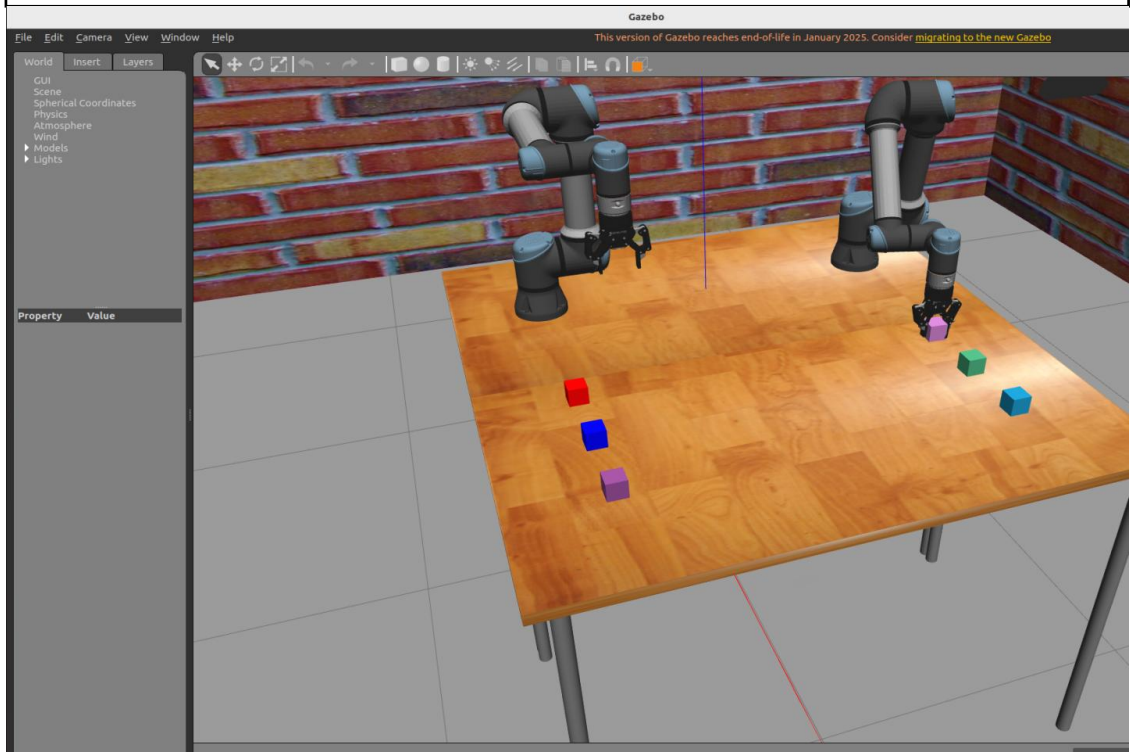
```
return LaunchDescription([
    Node(
        package='ur5e_gripper_control',
        # namespace='left_ur5e',
        executable='demo1',
        name='demo1_left_ur5e_node',
        parameters=[{
            "target_position_1": [0.5, -0.3, 0.4],
            "target_position_2": [0.5, -0.3, 0.15],
            "target_grasp_angle": 0.42,
            "target_position_3": [0.5, 0.0, 0.4],
            "use_sim_time":True,
        },
        robot_description_kinematics
        ]
    ),
])
```

```
return LaunchDescription([
    Node(
        package='ur5e_gripper_control',
        # namespace='left_ur5e',
        executable='demo1',
        name='demo1_left_ur5e_node',
        parameters=[{
            "target_position_1": [0.5, 0.0, 0.4],
            "target_position_2": [0.5, 0.0, 0.15],
            "target_grasp_angle": 0.42,
            "target_position_3": [0.5, -0.3, 0.4],
            "use_sim_time":True,
        },
        robot_description_kinematics
        ]
    ),
])
```

Task 3 Modify the trajectory of Demo 1 to pick up the cube

- Launch Demo 1 again.

```
ros2 launch ur5e_gripper_control demo1.launch.py
```



Task 4 Impove Demo 1: Place the cube, open the gripper and retract.

- Open the project folder in VS Code
- Open /home/qtfuture/UR5e_Vision_Assemble/src/ur5e_gripper_control/src/demo1.cpp
- Add Step 4,5,6 and 7 after “std::this_thread::sleep_for(std::chrono::seconds(2)); // wait for grasping” line 149

Step 4 & 5

```
// step 4
// move to target pose 3
success = manipulator_plan_and_execute(move_group, target_position_3, {0, M_PI, M_PI_2});
if(!success){
  rclcpp::shutdown();
  return 0;
}

// step 5: move down to place
std::vector<double> place_position = target_position_3;
place_position[2] = 0.16;
RCLCPP_INFO(LOGGER, "Lowering arm to place the object...");
success = manipulator_plan_and_execute(move_group, place_position, {0, M_PI, M_PI_2});
if(!success){
  rclcpp::shutdown();
  return 0;
}
```

Task 4 Impove Demo 1: Place the cube, open the gripper and retract.

Step 6 & 7

```
// step 6: open the gripper
goal_msg.command.position = 0.0;
RCLCPP_INFO(LOGGER, "Opening gripper to release object");
gripper_client->async_send_goal(goal_msg, send_goal_options);
std::this_thread::sleep_for(std::chrono::seconds(2));

// step 7: retract
std::vector<double> retract_position = target_position_3;
retract_position[2] += 0.15;
success = manipulator_plan_and_execute(move_group, retract_position, {0, M_PI, M_PI_2});
```

- “ctrl+c” in 2 terminals
- rebuild the repo

```
colcon build --symlink-install
```

Task 4 Impove Demo 1: Place the cube, open the gripper and retract.

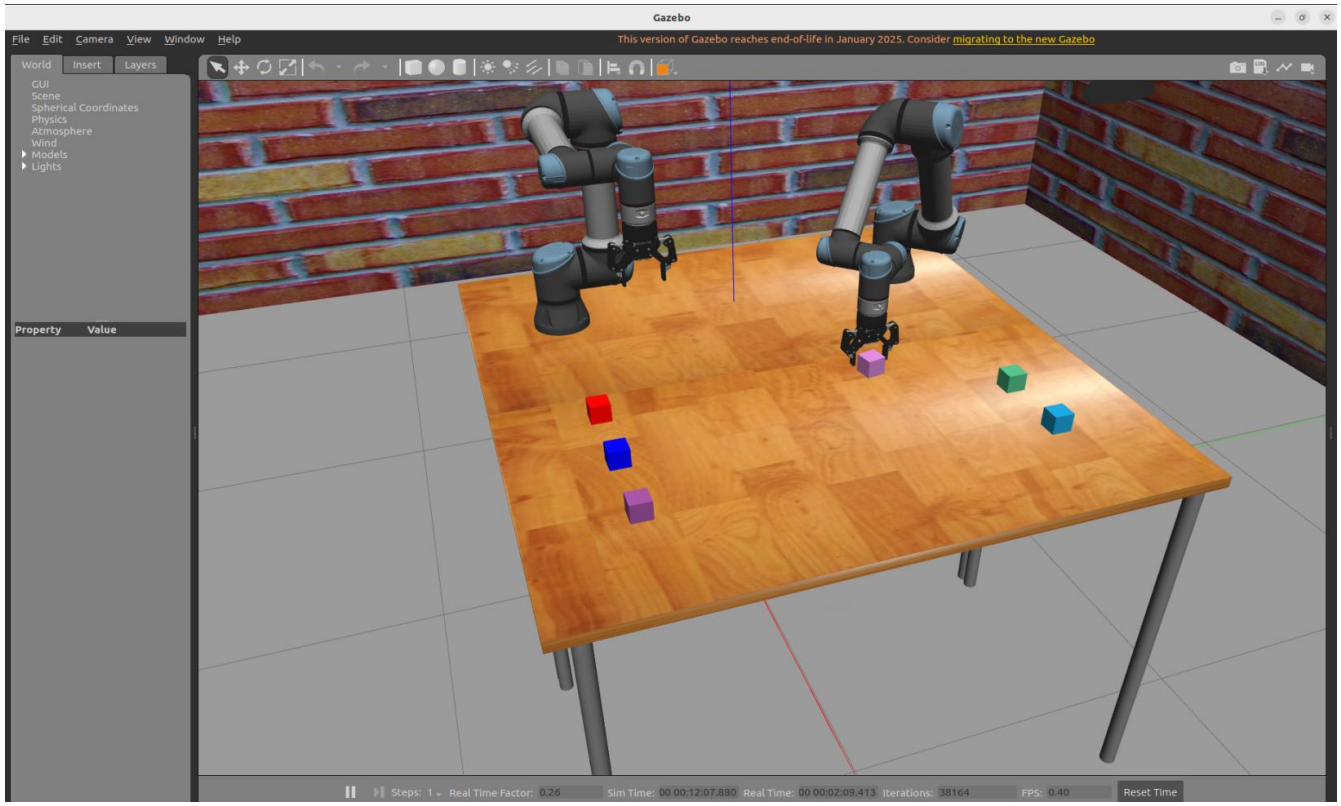
Terminal 1

```
ros2 launch bringup simulation.launch.py
```

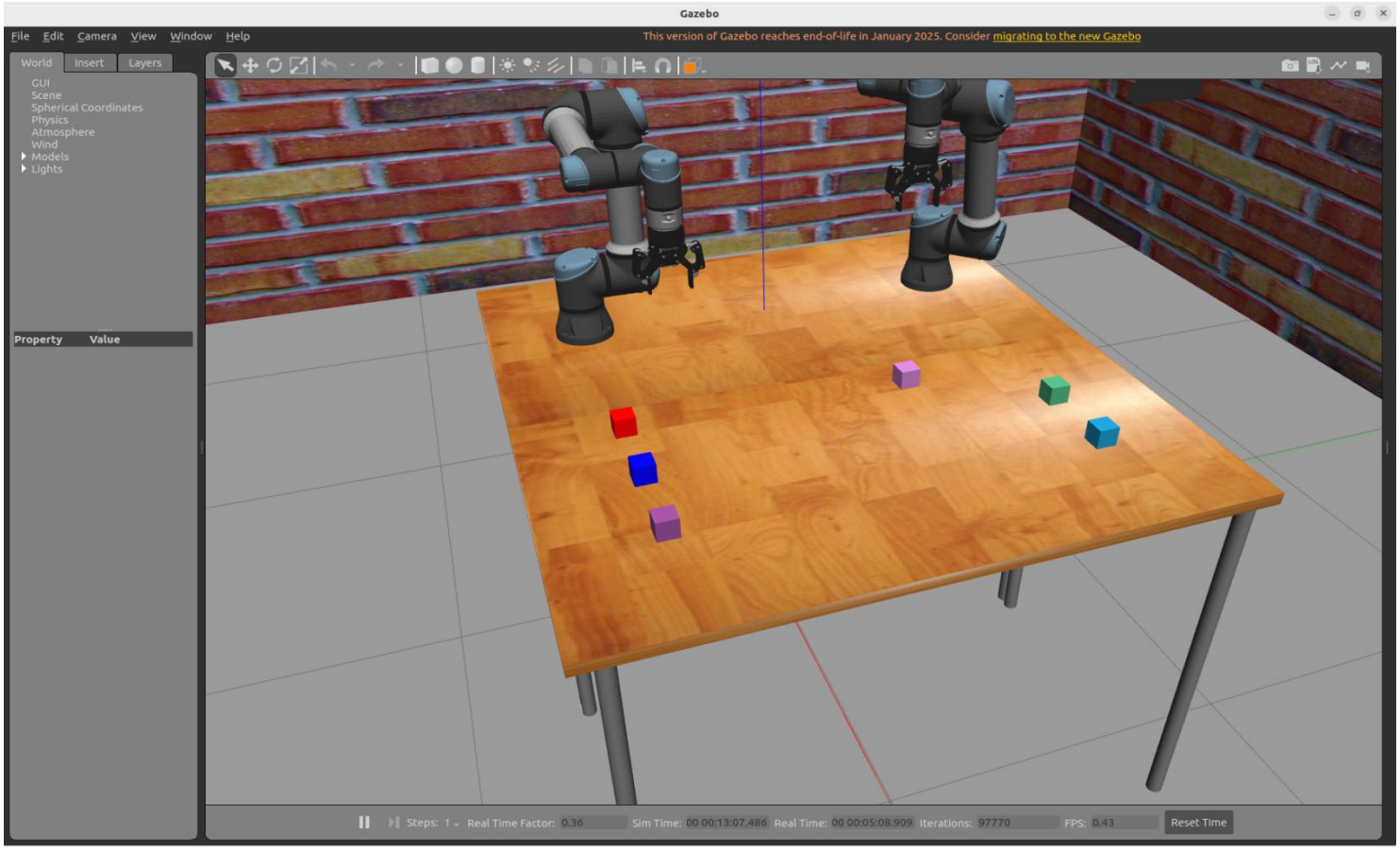
Terminal 2

```
ros2 launch ur5e_gripper_control demo1.launch.py
```

Task 4 Impove Demo 1: Place the cube, open the gripper and retract.



Task 4 Impove Demo 1: Place the cube, open the gripper and retract.



Task 5: Use right arm to do the same thing

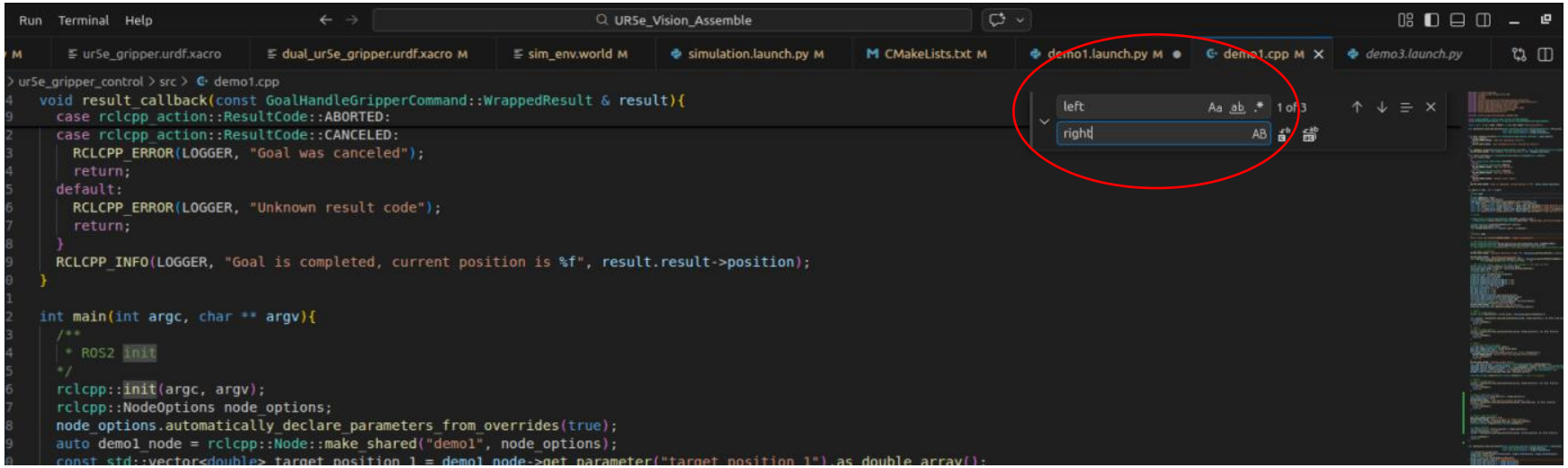
- Open the project folder in VS Code
- Open src/ur5e_gripper_control/launch/demo1.launch.py
- Modify the target_position value and name as shown in the figure below.

```
return LaunchDescription([
    Node(
        package='ur5e_gripper_control',
        # namespace='left_ur5e',
        executable='demo1',
        name='demo1_left_ur5e_node',
        parameters=[{
            "target_position_1": [0.5, 0.0, 0.4],
            "target_position_2": [0.5, 0.0, 0.15],
            "target_grasp_angle": 0.42,
            "target_position_3": [0.5, -0.3, 0.4],
            "use_sim_time":True,
        },
```

```
return LaunchDescription([
    Node(
        package='ur5e_gripper_control',
        # namespace='right_ur5e',
        executable='demo1',
        name='demo1_right_ur5e_node',
        parameters=[{
            "target_position_1": [0.5, 0.0, 0.4],
            "target_position_2": [0.5, 0.0, 0.15],
            "target_grasp_angle": 0.42,
            "target_position_3": [0.5, 0.3, 0.4],
            "use_sim_time":True,
        },
```

Task 5: Use right arm to do the same thing

- Open /home/qtfuture/UR5e_Vision_Assemble/src/ur5e_gripper_control/src/demo1.cpp
- Replace left with right

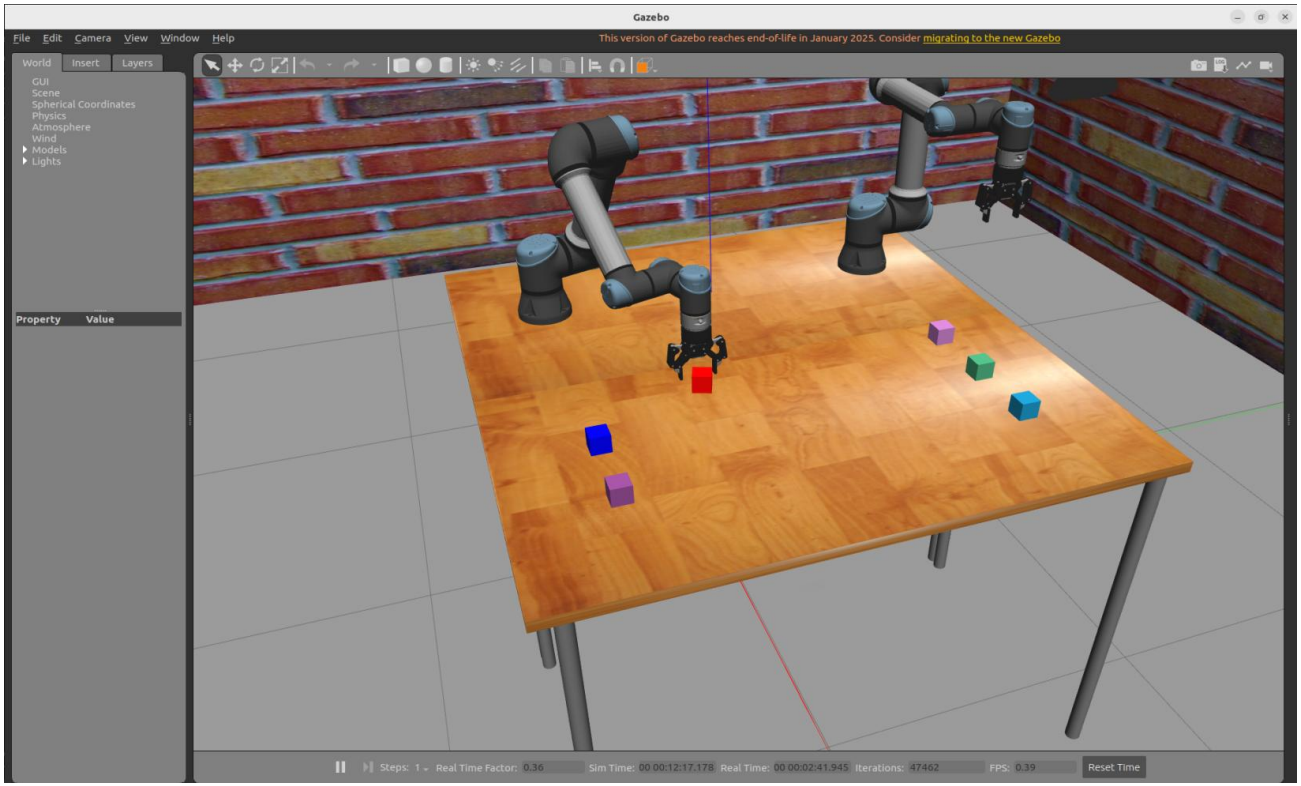


- rebuild the repo in terminal 2

```
colcon build --symlink-install
```

Task 5: Use right arm to do the same thing

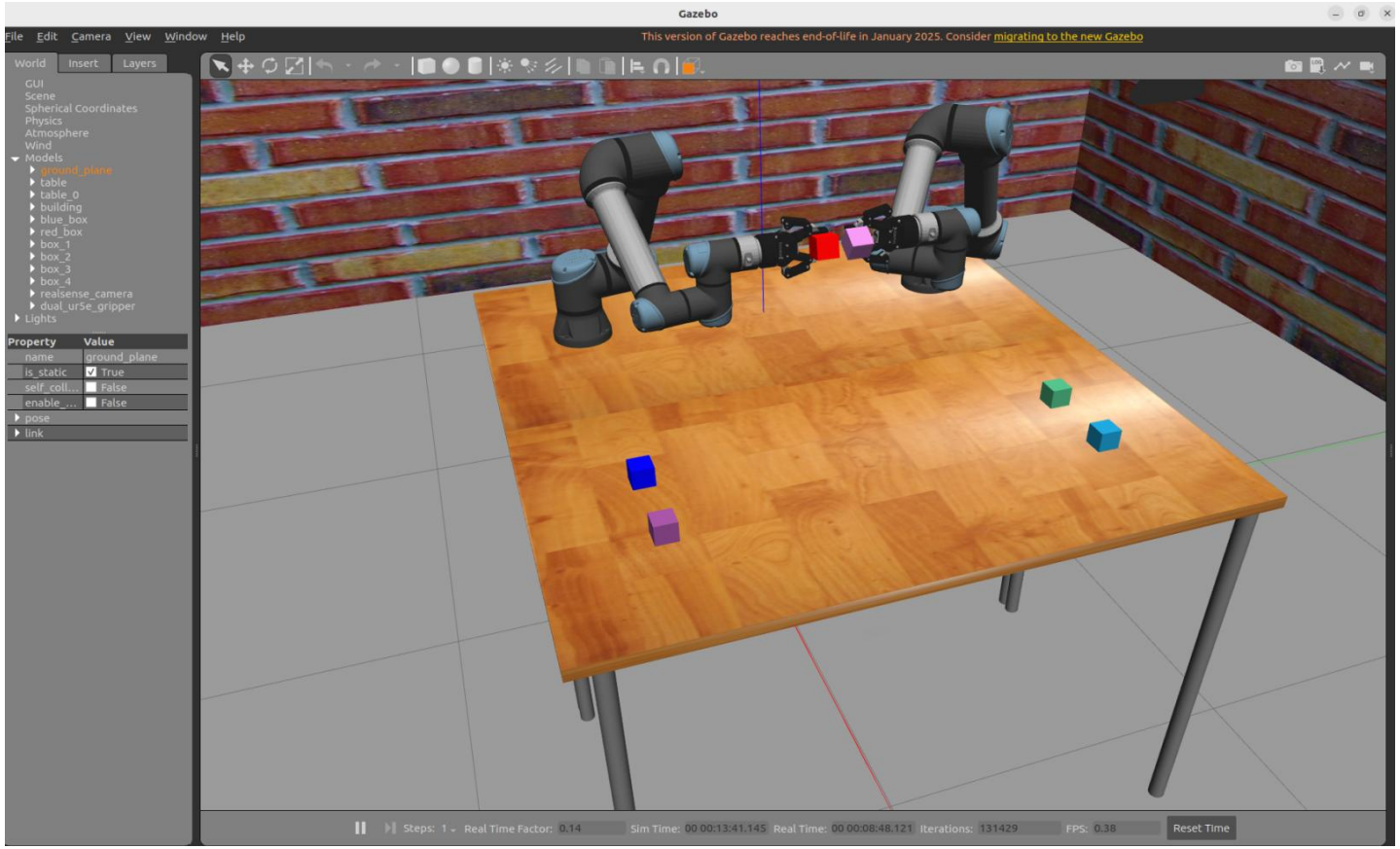
```
ros2 launch ur5e_gripper_control demo1.launch.py
```



Task 6 Deploy Demo 2:

```
ros2 launch ur5e_gripper_control demo2.launch.py
```

The demo2 setup demonstrates concurrent dual-arm operation by launching two independent nodes that simultaneously control the left and right arms to execute their own distinct pick-and-move sequences.



Task 7 Deploy Demo 3:

- "ctrl+c" in 2 terminals

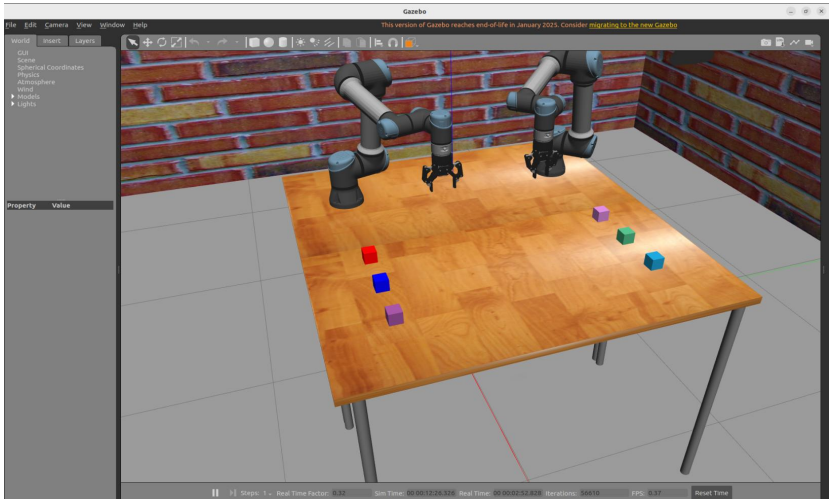
Terminal 1

```
ros2 launch bringup simulation.launch.py
```

Terminal 2

```
ros2 launch ur5e_gripper_control demo3.launch.py
```

- The demo3 node demonstrates synchronized dual-arm motion by combining individual inverse kinematics solutions into a single joint-space plan, executing the simultaneous movement.



Thank you!

Acknowledge:
We thank Javier Esteban, Ardit Ramadani, Fahrid Azampour and Zhongliang Jiang for making the original version of the slides